

Python Loops: A Reference

Loops allow you to automate repetitive tasks by running a block of code multiple times. Python features two main types: **for** loops (for iterating over collections or known sequences) and **while** loops (for repeating as long as a condition stays **True**).

1. Core Loop Structures

Loop Type	Instruction & Core Mechanics	Step-by-Step Code & Output
for item in item_list:	For Loop: Automatically pulls one item at a time from a collection (like a list or string), assigns it to a temporary variable, runs the indented code block, and repeats until the list finishes.	<pre>>>> items = ["apple", "pear"] >>> for item in items: ... print(f"I want a {item}") ... I want a apple I want a pear</pre>
while condition:	While Loop: Continuously executes the indented block as long as its condition evaluates to True . Ensure the loop code eventually alters the condition to avoid an infinite loop crash!	<pre>>>> count = 1 >>> while count <= 3: ... print(count) ... count = count + 1 # Update counter ... 1 2 3</pre>

2. Numerical Iteration with range()

The **range()** function generates a sequence of numbers, acting as a structural blueprint for **for** loops when you don't have a pre-existing list.

Syntax	Instruction & Core Mechanics	Step-by-Step Code & Output
range(stop)	Generates numbers starting from 0 up to, but not including , the specified stop number.	<pre>>>> for i in range(3): ... print(i) ... 0 1 2</pre>
range(start, stop)	Generates numbers starting explicitly from the start value up to, but not including , the stop value.	<pre>>>> for i in range(5, 8): ... print(i) ... 5 6 7</pre>

Syntax	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>range(start, stop, step)</code>	Generates a sequence skipping numbers based on the <code>step</code> value. Can count forwards or backwards.	<pre>>>> for i in range(0, 7, 2): ... print(i) ... 0 2 4 6</pre>

3. Loop Control Statements

Keywords used inside loops to manipulate execution behavior dynamically based on conditional checks.

Keyword	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>break</code>	Immediately terminates the entire loop execution and jumps to the next unindented line outside the loop structure.	<pre>>>> for num in range(1, 10): ... if num == 4: ... break # Exits loop immediately ... print(num) ... 1 2 3</pre>
<code>continue</code>	Skips the rest of the current iteration block and jumps straight back to the top of the loop for the next cycle.	<pre>>>> for num in range(1, 5): ... if num == 3: ... continue # Skips printing 3 ... print(num) ... 1 2 4</pre>